Amazon Web Services

Containers
Complete Learning Guide

Chapter 1 — Amazon Elastic Container Registry (ECR)

Chapter 2 — Amazon Elastic Container Service (ECS)

Chapter 3 — Amazon Elastic Kubernetes Service (EKS)

Introduction | Core Concepts | Architecture | Features | Use Cases | Integrations | Comparisons

Beginner-Friendly • In-Depth • Practical

Table of Contents

Each chapter follows the same structure: Simple Introduction, Real-World Analogy, Core Concepts & Terminology, Architecture & Workflow, Key Features, Real-World Use Cases, AWS Integrations, and Comparison Tables.

Chapter 1 — Amazon Elastic Container Registry (ECR)

Fully Managed Docker Container Image Registry

1. Simple Introduction

Amazon ECR (Elastic Container Registry) is a fully managed container image registry that makes it easy to store, manage, share, and deploy container images and OCI (Open Container Initiative) artifacts. It eliminates the need to operate your own container repository infrastructure.

Before ECR, development teams had to set up and maintain their own Docker registries (like Docker Hub or a self-hosted Harbor), manage storage, handle authentication, and ensure high availability. ECR removes all of that operational burden.

Analogy — The Software Library

Think of ECR like a private library for your application blueprints (container images). When a developer builds a new version of an app, they 'check in' the blueprint to the library (push the image). When a server needs to run that app, it visits the library and 'checks out' the exact version it needs (pull the image). The library is secure, always available, and automatically organised.

Business Problem ECR Solves

Store Docker/OCI images privately without managing a registry server

Secure images with IAM — only authorised services and users can pull images

Seamless integration with ECS, EKS, and Lambda — no extra credentials needed

Scan images automatically for OS and software vulnerabilities (CVEs)

Replicate images across AWS Regions for global deployments

2. Core Concepts & Terminology

Term	Definition	Real-World Equivalent
Container Image	A read-only, self-contained package with app code, runtime, libraries, and config	An ISO disk image of a fully configured OS + app
Registry	The top-level storage service that holds one or more repositories	Docker Hub / GitHub Packages (but private and on AWS)
Repository	A named collection of related container images (one per app/service)	A Git repository, but for container images

Term	Definition	Real-World Equivalent
Image Tag	A human-readable label attached to an image version (e.g. 'latest', 'v1.2.3')	A Git tag / branch name
Image Digest	An immutable SHA-256 hash uniquely identifying an exact image	A Git commit hash (cannot change)
Push	Uploading a locally-built image to ECR	git push
Pull	Downloading an image from ECR to a runtime environment	git pull / git clone
Lifecycle Policy	Rules that automatically delete old or unused image versions to save cost	Auto-archive old files after N days
Image Scanning	Automated CVE vulnerability scanning of image layers	Running an antivirus / security audit on code
Replication	Copy images to other AWS Regions or accounts automatically	CDN caching of assets across locations
OCI Artifact	Non-image content stored in ECR: Helm charts, WASM modules, SBOMs	Storing configuration files alongside code
Cross-Account Access	Allow another AWS account to pull images via resource-based policy	Sharing a private repo with a partner

Registry Types

Type	Description	URL Format	Use Case
Private Registry	Default. Images only accessible with IAM auth.	ACCOUNT_ID.dkr.ecr.REGION.amazonaws.com	Internal apps, sensitive workloads
Public Registry	ECR Public Gallery. Anyone can pull without auth.	public.ecr.aws/alias/repo	Open-source base images, public tools

3. Architecture & Workflow

End-to-End Container Image Lifecycle

ECR Workflow

DEVELOPER MACHINE

1. docker build -t my-api:v2.0 . <- Build image locally
2. aws ecr get-login-password | docker login <- Authenticate to ECR
3. docker tag my-api:v2.0 ACCOUNT.dkr.ecr.REGION.amazonaws.com/my-api:v2.0
4. docker push ACCOUNT.dkr.ecr.REGION.amazonaws.com/my-api:v2.0

ECR SERVICE

5. Stores image layers in S3 (managed, encrypted with KMS)
6. Runs vulnerability scan (Basic or Enhanced/Inspector)
7. Lifecycle policy checks: delete images older than 30 days if untagged
8. Replication rule: copy image to us-west-2 and eu-west-1

DEPLOYMENT (ECS / EKS / Lambda)

9. ECS Task Definition references: ACCOUNT.dkr.ecr.REGION.amazonaws.com/my-api:v2.0
10. ECS pulls image from ECR (no password needed — uses IAM role)
11. Container starts and serves traffic

4. Key Features

Feature	Description	Business Value
Fully Managed	AWS handles storage, availability, patching, and scaling	Zero registry infrastructure to operate
IAM Integration	Push/pull permissions controlled via IAM policies and resource-based policies	Fine-grained, auditable access control
Image Scanning — Basic	Free CVE scan on push using open-source Clair engine	Detect known OS vulnerabilities automatically
Image Scanning — Enhanced	Powered by Amazon Inspector; scans OS packages AND application packages	Deeper security posture for compliance
Lifecycle Policies	Rules to auto-expire images (e.g. keep only last 10 tagged, delete untagged > 7 days)	Automated storage cost reduction
Cross-Region Replication	Automatically replicate images to other regions	Faster pulls for global deployments
Cross-Account Replication	Replicate to another AWS account (e.g. dev -> prod account)	Multi-account CI/CD pipelines
Immutable Tags	Prevent overwriting an existing image tag	Enforce immutable release artifacts
Encryption	Images encrypted at rest with KMS (customer-managed keys supported)	Security & compliance
Pull-Through Cache	Cache public images (Docker Hub, ECR Public) into your private ECR	Reduce Docker Hub rate-limit issues in CI/CD

Feature	Description	Business Value
OCI Artifact Support	Store Helm charts, SBOM files, and WASM modules alongside images	Single registry for all container artifacts
Private Endpoints (PrivateLink)	Pull images over private VPC network without going to the internet	Security — no public internet exposure

5. Real-World Use Cases

Industry / Scenario	How ECR Is Used	Key ECR Feature
E-Commerce (Microservices)	Each microservice (cart, payment, inventory) has its own ECR repo; CI/CD pushes on every merge to main	Per-service repos + lifecycle policies
Banking / Fintech	All images scanned for CVEs before promotion to production; immutable tags enforce audit trail	Enhanced scanning + immutable tags
Healthcare (HIPAA)	Images encrypted with customer-managed KMS keys; private endpoints — no internet egress	KMS encryption + PrivateLink
Global SaaS Platform	Image built once in us-east-1, replicated to ap-south-1 and eu-west-1 automatically for fast regional pulls	Cross-region replication
Multi-Account Org	Shared 'golden base image' repo in a central security account; dev/prod accounts pull via cross-account policy	Cross-account replication
CI/CD Pipeline (GitHub Actions)	Build Docker image, push to ECR, trigger ECS/EKS rolling update	IAM OIDC + pull-through cache
ML Model Serving	Store custom SageMaker inference container images in ECR	OCI artifact storage

6. AWS Service Integrations

AWS Service	How It Integrates	Use Case
Amazon ECS	Task definitions reference ECR image URIs; ECS pulls automatically using task IAM role	Deploy containers without credentials
Amazon EKS	Pod specs reference ECR URIs; node IAM role grants pull access	Kubernetes workloads use private images
AWS Lambda	Deploy Lambda functions as container images (up to 10 GB) stored in ECR	Large ML model inference as Lambda
AWS CodePipeline / CodeBuild	CodeBuild builds and pushes images to ECR; CodePipeline triggers ECS deploy	Fully automated CI/CD on AWS
Amazon Inspector	Enhanced scanning: Inspector analyses image layers for CVEs continuously	Ongoing vulnerability management
AWS IAM	Resource-based repo policies + IAM identity policies control access	Zero-trust access control
AWS KMS	Encrypt image layers with customer-managed KMS keys	Compliance encryption

AWS Service	How It Integrates	Use Case
Amazon CloudWatch	Metrics on push/pull counts, image scan findings	Operational visibility
AWS CloudTrail	Audit every push, pull, delete, and policy change	Security audit trail
AWS PrivateLink	VPC endpoint for ECR API and Docker registry — no internet needed	Secure private network access

7. ECR vs Other Container Registries

Feature	Amazon ECR	Docker Hub	GitHub GHCR	Self-Hosted Harbor
Managed	Fully managed	Managed	Managed	You manage
Auth	AWS IAM	Username/token	GitHub token	LDAP / OIDC
Private Repos	Unlimited	1 free	Unlimited	Unlimited
Vulnerability Scan	Built-in	Paid add-on	Basic (Trivy)	Built-in
AWS Integration	Native	Manual	Manual	Manual
Data Residency	Your AWS region	Docker infra	GitHub infra	Your servers
Pull-Through Cache	Yes	N/A	N/A	Yes
OCI Artifacts	Yes	Yes	Yes	Yes
Cost Model	Pay per GB+data transfer	Subscription	Included w/ GitHub	Infrastructure cost

When to Choose ECR

You are already on AWS and deploying to ECS, EKS, or Lambda

You need IAM-based access control and AWS CloudTrail auditing

You require images to stay within your AWS account and region

You need cross-region or cross-account image replication automatically

Consider Docker Hub if you have open-source public images you want the community to pull

Chapter 2 — Amazon Elastic Container Service (ECS)

AWS-Native Fully Managed Container Orchestration Service

1. Simple Introduction

Amazon ECS (Elastic Container Service) is a fully managed container orchestration service that allows you to run, stop, and manage Docker containers on a cluster. It is AWS's own container orchestrator — built specifically for AWS and tightly integrated with the entire AWS ecosystem.

Container orchestration solves the problem of managing many containers across many servers: Where should each container run? What happens when a container crashes? How do you update containers without downtime? How do you scale up when traffic spikes? ECS answers all of these questions automatically.

Analogy — The Restaurant Manager

Imagine your app is a restaurant. Each chef (container) prepares one dish. ECS is the restaurant manager: it decides how many chefs to hire (scale), which kitchen table to assign each chef to (scheduling), replaces a chef who gets sick (auto-recovery), and re-trains a new chef when the menu changes (rolling updates). You just describe what dishes need to be served — ECS handles the staffing.

Business Problem ECS Solves

Run containers at scale without managing a Kubernetes control plane

Automatically replace crashed containers and maintain desired state

Scale containers up/down based on CPU, memory, or custom metrics

Load-balance traffic across healthy containers automatically

Deploy new versions with zero downtime (rolling updates, blue/green)

Run containers serverlessly with AWS Fargate — no EC2 servers to manage

2. Core Concepts & Terminology

Term	Definition	Analogy
Cluster	Logical grouping of infrastructure (EC2 instances or Fargate capacity) where your tasks and services run	The restaurant building / kitchen space

Term	Definition	Analogy
Task Definition	A blueprint (JSON) describing one or more containers: image, CPU, memory, ports, env vars, volumes, IAM role	A recipe card specifying ingredients and method
Task	A running instance of a Task Definition — one or more containers running together	A dish being actively prepared
Service	Ensures a specified number of Tasks are always running; handles load balancing and rolling updates	The manager ensuring 5 chefs are always working
Container Definition	The per-container section inside a Task Definition (image URI, ports, env vars)	Individual chef's assignment
Launch Type	How tasks run: EC2 (you manage servers) or Fargate (serverless, AWS manages servers)	Own kitchen vs renting a commercial kitchen per hour
Fargate	Serverless compute for ECS/EKS — no EC2 instances to provision or manage	On-demand kitchen rental (pay per dish, not per kitchen)
Task IAM Role	IAM role assigned to containers in a task — grants AWS API permissions	Chef's access badge to different pantry rooms
Task Execution Role	IAM role used by ECS agent to pull images from ECR and publish logs to CloudWatch	HR department's authority to hire and onboard
Service Discovery	Automatic DNS registration for services using AWS Cloud Map	Internal company phone directory
Capacity Provider	Strategy for mixing Fargate and EC2 (Spot or On-Demand) for cost optimisation	Choosing between freelance chefs and full-time staff
ECS Anywhere	Run ECS tasks on your own on-premises servers, not just AWS	Using the restaurant manager to coordinate remote catering

EC2 Launch Type vs Fargate Launch Type

Dimension	EC2 Launch Type	Fargate Launch Type
Server Management	You provision, patch, and scale EC2 instances	AWS manages all compute
Cost Model	Pay for EC2 instances (running or idle)	Pay per vCPU/memory per second
Control	Full OS-level control, custom AMI, GPU support	No OS access; only container-level
Startup Speed	Faster (containers start on existing hosts)	Slightly slower (infrastructure provision)
Best For	Long-running, predictable, high-density workloads; GPU workloads	Variable traffic, microservices, batch jobs
Cost Optimisation	Use Spot Instances for up to 90% savings	Fargate Spot for up to 70% savings

3. Architecture & Working

ECS Architecture Overview

ECS Architecture (Fargate Launch Type)

USER / INTERNET

|

[Application Load Balancer] <-- distributes HTTP/HTTPS traffic

|

[ECS SERVICE]

| Maintains desired count (e.g. 3 tasks always running)

| Handles rolling deployment on new Task Definition version

| Registers/deregisters tasks with ALB Target Group

|

+--[TASK 1: Fargate]--+--[TASK 2: Fargate]--+--[TASK 3: Fargate]

[Container A] [Container A] [Container A]

[Container B] [Container B] [Container B]

(sidecar) (sidecar) (sidecar)

SUPPORTING SERVICES:

- ECR: stores the container images pulled at task start
- CloudWatch Logs: receives container stdout/stderr
- AWS Secrets Manager: injects secrets as env vars
- AWS IAM: Task Role grants containers access to S3, DynamoDB, etc.
- AWS Cloud Map: service discovery DNS for inter-service calls

Step-by-Step: How ECS Deploys a New Version

Step	Action	Detail
1	Developer pushes new image	CI/CD pipeline builds image and pushes to ECR with tag v2.0
2	New Task Definition registered	Register task definition revision pointing to new image tag
3	Update ECS Service	aws ecs update-service --task-definition my-app:2 --force-new-deployment
4	ECS starts new tasks	ECS launches new tasks (Fargate provisions new compute automatically)
5	Health check passes	ALB target group health check confirms new containers are healthy
6	Traffic gradually shifted	ALB drains connections from old tasks, sends traffic to new tasks
7	Old tasks stopped	ECS stops old task versions after connection draining completes
8	Rollback if needed	Update service to previous Task Definition revision — ECS repeats process

4. Key Features

Feature	Description	Benefit
Fargate (Serverless)	Run containers without managing EC2 instances	Zero server ops; pay per task second
Service Auto Scaling	Scale tasks based on CPU, memory, ALB request count, or custom CloudWatch metrics	Handle traffic spikes automatically
Rolling Updates	Deploy new version gradually, keeping old version running during transition	Zero-downtime deployments
Blue/Green Deployment	Via CodeDeploy: run old (blue) and new (green) simultaneously, switch instantly	Instant rollback capability
ALB Integration	Native integration with Application Load Balancer for path and host routing	Route /api/* to one service, /web to another
Service Discovery	Automatic DNS via AWS Cloud Map for service-to-service communication	No hard-coded IPs between microservices
ECS Exec	Interactive shell into a running container for debugging	Production debugging without SSH
Task Placement Strategies	Spread, binpack, or random placement of tasks across EC2 instances	Optimise cost and availability
Secrets Integration	Inject Secrets Manager and Parameter Store values as env vars at task start	No secrets in code or images
Spot Integration	Run tasks on Fargate Spot or EC2 Spot for up to 90% cost reduction	Cost-efficient batch and non-critical workloads
ECS Anywhere	Extend ECS to on-premises or other cloud servers	Hybrid cloud container management
Container Health Checks	Define health checks per container; unhealthy containers auto-replaced	Self-healing applications

5. Real-World Use Cases

Industry	Use Case	ECS Configuration
E-Commerce	Run product, cart, and payment microservices as separate ECS services; scale each independently on peak days	Fargate + ALB + Service Auto Scaling
Fintech	Process financial transactions in isolated Fargate tasks; each task gets a fresh environment per request	Fargate + Secrets Manager + VPC isolation
Media & Streaming	Video transcoding jobs triggered on S3 upload; each job is a separate ECS task that auto-terminates	EC2 Spot + SQS trigger + S3
Healthcare (HIPAA)	Patient data processing in isolated Fargate tasks with VPC endpoints; no internet egress	Fargate + PrivateLink + KMS
Gaming	Game session servers on EC2 launch type for GPU support; lobby service on Fargate	EC2 GPU (g4dn) + Fargate hybrid
Startups / SaaS	Entire backend on Fargate — no servers to manage; scale from 0 to millions of users	Fargate + ALB + RDS Proxy

Industry	Use Case	ECS Configuration
Data Pipelines	ETL tasks triggered on schedule or by events; containers start, process, and terminate	Fargate Spot + EventBridge + S3

6. AWS Service Integrations

AWS Service	Integration	Use Case
Amazon ECR	Pull container images using task execution IAM role	Private image storage
ALB / NLB	Register tasks as targets; path/host-based routing	Load balance and route traffic
AWS Fargate	Serverless compute engine for ECS tasks	No-server container execution
Amazon CloudWatch	Container logs (awslogs driver) + metrics + alarms	Monitoring and alerting
AWS X-Ray	Distributed tracing across microservices	Request tracing and debugging
AWS Secrets Manager	Inject secrets as environment variables at task launch	Secure credential delivery
AWS IAM	Task roles for fine-grained AWS API access	Container-level permissions
AWS CodeDeploy	Blue/green deployments for ECS services	Zero-downtime deploys
AWS Cloud Map	DNS-based service discovery between services	Microservice communication
Amazon EFS	Mount shared persistent file storage into containers	Stateful workloads
AWS App Mesh	Service mesh with mTLS and traffic control via Envoy sidecar	Advanced traffic management
Amazon SQS	Trigger task scale-out based on queue depth	Worker pool pattern

7. ECS vs EKS vs Docker Swarm

Feature	Amazon ECS	Amazon EKS	Docker Swarm
Orchestrator	AWS proprietary	Kubernetes (CNCF)	Docker native
Learning Curve	Low	High	Very Low
AWS Integration	Deep native	Deep (managed)	Manual
Portability	AWS-only	Kubernetes (portable)	Docker hosts
Serverless Option	Fargate (native)	Fargate (limited)	None
Ecosystem	AWS services	Vast K8s ecosystem	Limited
Control Plane Cost	Free	$0.10/hr per cluster	Free

Feature	Amazon ECS	Amazon EKS	Docker Swarm
Best For	AWS-native, simple container orchestration	Complex workloads, K8s portability	Simple local dev / small teams

When to Choose ECS over EKS

Your team is small and wants simplicity over Kubernetes flexibility

You are 100% on AWS and do not need to run the same workload on GKE or AKS

You want Fargate (serverless) as the primary compute — ECS Fargate is simpler

You are migrating from a non-containerised architecture and want a gentle ramp-up

Choose EKS when: you need Kubernetes APIs, helm charts, K8s RBAC, or must run workloads portably across cloud providers

Chapter 3 — Amazon Elastic Kubernetes Service (EKS)

Fully Managed Kubernetes Control Plane on AWS

1. Simple Introduction

Amazon EKS (Elastic Kubernetes Service) is a fully managed Kubernetes service that makes it easy to run Kubernetes on AWS without needing to install, operate, and maintain your own Kubernetes control plane or nodes.

Kubernetes (K8s) is the industry-standard open-source container orchestration system created by Google and now maintained by the CNCF (Cloud Native Computing Foundation). It automates deploying, scaling, and managing containerised applications. EKS removes the hardest part of Kubernetes — running the control plane — and lets you focus on deploying your applications.

Analogy — Air Traffic Control

Kubernetes is like an air traffic control system managing hundreds of planes (containers) across many runways (nodes/servers). The control tower (control plane) tracks every plane, assigns landing slots, handles emergencies (crashes), and coordinates new arrivals. Running your own Kubernetes is like building and staffing the control tower yourself. EKS gives you a pre-built, 24x7 staffed control tower — you only manage the planes and runways.

Business Problem EKS Solves

Run production Kubernetes without managing etcd, kube-apiserver, scheduler, controller-manager

Kubernetes control plane is automatically highly available across 3 AZs

Native integration with AWS networking (VPC CNI), IAM, ALB, and CloudWatch

Run Kubernetes worker nodes as EC2 (self-managed), Managed Node Groups, or Fargate

Use the vast Kubernetes ecosystem: Helm, Istio, Argo CD, Prometheus, Grafana

Portability: same YAML manifests run on EKS, GKE, AKS, or on-premises

2. Core Concepts & Terminology

Kubernetes Fundamentals (Required Background)

K8s Term	Definition	ECS Equivalent
Pod	Smallest deployable unit in K8s; one or more containers sharing network/storage	ECS Task

K8s Term	Definition	ECS Equivalent
Node	A worker machine (EC2 instance) that runs Pods	EC2 instance in ECS cluster
Cluster	The complete K8s environment: control plane + worker nodes	ECS Cluster
Deployment	Manages a set of identical Pods; handles rolling updates and rollbacks	ECS Service
Service (K8s)	Stable network endpoint (DNS + load balancing) in front of a set of Pods	ECS Service + Target Group
Namespace	Virtual cluster within a cluster for resource isolation	N/A (ECS has no equivalent)
ConfigMap	Key-value store for non-secret configuration injected into Pods	ECS environment variables
Secret	Base64-encoded sensitive data (passwords, tokens) injected into Pods	Secrets Manager reference in Task Def
Ingress	HTTP/HTTPS routing rules; maps hostnames/paths to K8s Services	ALB Listener Rules
PersistentVolume (PV)	Storage resource (EBS, EFS) that lives independently of Pods	ECS Volume
HPA (Horizontal Pod Autoscaler)	Automatically scales Pod count based on CPU/memory/custom metrics	ECS Service Auto Scaling
DaemonSet	Ensures one Pod runs on every node (e.g. log collector, monitoring agent)	No direct ECS equivalent
StatefulSet	Manages stateful Pods with stable identities (e.g. databases)	No direct ECS equivalent
Helm Chart	Package manager for K8s — bundles all manifests for an app into a reusable chart	No ECS equivalent

EKS-Specific Concepts

EKS Term	Definition
Managed Node Group	AWS provisions and lifecycle-manages EC2 worker nodes; auto-updates, auto-repair
Self-Managed Nodes	You launch EC2 instances yourself and join them to the EKS cluster; full control
EKS Fargate Profile	Run Pods on serverless Fargate — no node management at all
EKS Anywhere	Run EKS on your own on-premises hardware using the same EKS APIs
AWS VPC CNI	Default CNI plugin; assigns real VPC IP addresses to Pods (not overlay network)
IRSA (IAM Roles for Service Accounts)	Assign IAM roles to Kubernetes Service Accounts — fine-grained Pod-level AWS access
EKS Add-ons	AWS-managed components: CoreDNS, kube-proxy, VPC CNI, EBS CSI driver — auto-updated

EKS Term	Definition
Karpenter	Open-source node auto-provisioner — launches the right EC2 instance for each Pod instantly
eksctl	CLI tool to create and manage EKS clusters with a single command

3. Architecture & Working

EKS Control Plane Architecture

EKS Architecture Deep Dive

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ EKS CONTROL PLANE (Managed by AWS) ■
■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■
■
■ ■ kube-apiserver■ ■kube-controller- ■ ■ kube-scheduler ■ ■
■ ■ (HTTPS 443) ■ ■ manager ■ ■ (places pods on ■ ■
■ ■ entry point ■ ■ (reconciles ■ ■ nodes) ■ ■
■ ■ for kubectl ■ ■ desired state) ■ ■■■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■ ■
■ ■ etcd (distributed key-value store — cluster state) ■ ■
■ ■ Runs across 3 AZs — AWS manages replicas and backups ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
| kubelet (secure channel)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ WORKER NODES (Your VPC — EC2 or Fargate) ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■ NODE 1 (EC2) ■ ■ NODE 2 (EC2) ■ ■
■ ■ kubelet kube-proxy■ ■ kubelet kube-proxy■ ■
■ ■ [Pod A] [Pod B] ■ ■ [Pod C] [Pod D] ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
| |
[AWS ALB Ingress Controller] [Amazon EBS/EFS CSI Driver]
Provisions ALB per Ingress Provisions EBS volumes for PVCs
```

Node Management Options Compared

Option	Who Manages Nodes	Customisation	Best For
Managed Node Groups	AWS provisions, patches, drains nodes; you choose instance type	Medium — custom AMI supported	Most production workloads; reduces ops burden

Option	Who Manages Nodes	Customisation	Best For
Self-Managed Nodes	You fully manage EC2 lifecycle, patching, and scaling	Maximum — any instance type, any AMI, GPU, bare metal	Specialised hardware needs; deep OS customisation
Fargate Profiles	AWS manages all compute; no nodes visible at all	None — container-level only	Serverless K8s; variable and bursty workloads
Karpenter (add-on)	Karpenter auto-provisions optimal EC2 instances per Pod request	High — consolidates and diversifies instance types	Cost-optimised, large dynamic clusters

4. Key Features

Feature	Description	Business Value
Managed Control Plane	AWS runs etcd, kube-apiserver, scheduler, controller-manager across 3 AZs	Zero control plane ops; SLA 99.95%
Automatic K8s Upgrades	Upgrade control plane with a click; managed node groups upgrade nodes automatically	Stay current without downtime risk
AWS VPC CNI	Pods get real VPC IPs — directly routable, no overlay NAT	Simpler networking; use Security Groups on Pods
IRSA	Pod-level IAM roles via OIDC — no node-level credentials needed	Least-privilege AWS access per service
EKS Add-ons	AWS manages CoreDNS, kube-proxy, VPC CNI, EBS CSI — keeps them patched	Critical components always up to date
ALB Ingress Controller	Automatically provisions AWS ALB for K8s Ingress resources	Native AWS load balancing for K8s
Secrets Store CSI Driver	Mount Secrets Manager / Parameter Store secrets as files into Pods	Secure secrets without K8s Secrets
EKS Anywhere	Run EKS on-premises or on other clouds with same tooling	Consistent hybrid cloud K8s
Karpenter Integration	Right-size EC2 nodes for Pods; bin-pack, consolidate, use Spot	Up to 60% compute cost savings
Windows Containers	Run Windows Server containers on EKS alongside Linux	Mixed Windows/.NET and Linux workloads
GPU & Inferentia Support	Run AI/ML workloads on GPU nodes or AWS Inferentia chips	ML training and inference on K8s
GitOps with Argo CD / Flux	EKS integrates with GitOps controllers for declarative deployments	Git as source of truth for deployments

5. Real-World Use Cases

Industry	Use Case	EKS Config Used
Large Enterprise SaaS	100+ microservices, multi-team, separate namespaces per team, GitOps with Argo CD, Istio service mesh	Managed Node Groups + IRSA + ALB Controller + Istio

Industry	Use Case	EKS Config Used
Banking / Financial	PCI-DSS compliant workloads; strict network policies; separate node groups for card processing vs analytics	Multiple node groups + K8s NetworkPolicy + Secrets CSI
E-Commerce (Peak Traffic)	Auto-scale from 50 to 5,000 Pods during sale events using HPA + Karpenter; use Spot Instances for 60% savings	Karpenter + HPA + Fargate Spot
AI / ML Platform	Train ML models on GPU node group; serve inference on Inferentia; KubeFlow for ML pipeline orchestration	GPU Managed Node Group + Karpenter + EFS
Ride-Sharing / Real-Time	Driver matching, surge pricing, mapping services each in separate K8s namespace; sub-10ms response using NLB	NLB + dedicated node groups + IRSA
Multi-Cloud / Hybrid	Same Helm charts deploy to EKS, GKE, and on-prem via EKS Anywhere; team uses one K8s skill set everywhere	EKS Anywhere + Helm + GitOps
Startup (Simple K8s)	Small team using eksctl + Fargate profile — zero node management; deploy with kubectl apply	Fargate profile + eksctl + ALB Ingress

6. AWS Service Integrations

AWS Service	Integration Mechanism	Use Case
Amazon ECR	Nodes pull images via IAM node/pod role; no credentials in Pod specs	Private container image storage
AWS ALB (via Ingress Controller)	K8s Ingress resources auto-provision Application Load Balancers	HTTP/S routing and TLS termination
AWS NLB (via Service type=LoadBalancer)	K8s Service of type LoadBalancer provisions Network Load Balancer	TCP/UDP load balancing
Amazon EBS (via CSI Driver)	PersistentVolumeClaims provision EBS gp3/io2 volumes	Stateful apps needing block storage
Amazon EFS (via CSI Driver)	Shared ReadWriteMany PVCs backed by EFS	Shared storage across multiple Pods
AWS Fargate	Fargate Profiles run select Pods serverlessly based on namespace/labels	Serverless K8s Pods
Amazon CloudWatch (Container Insights)	Collect K8s metrics and logs via CloudWatch agent DaemonSet	Cluster-wide observability
AWS X-Ray	SDK in app code + X-Ray DaemonSet for distributed tracing	Microservice request tracing
AWS Secrets Manager (CSI Driver)	Mount secrets as files in Pod filesystem at launch	Secure secret delivery to Pods
IRSA (IAM + OIDC)	K8s Service Account annotated with IAM role ARN; short-lived tokens	Fine-grained, per-Pod IAM access
AWS App Mesh	Envoy sidecar injected into Pods; App Mesh controls traffic rules	Service mesh, canary releases, mTLS

AWS Service	Integration Mechanism	Use Case
Amazon GuardDuty	EKS audit log monitoring for threat detection	K8s security threat detection

7. EKS vs ECS vs Self-Managed Kubernetes

Dimension	Amazon EKS	Amazon ECS	Self-Managed K8s (on EC2)
Orchestrator	Kubernetes (CNCF standard)	AWS proprietary	Kubernetes
Control Plane	Managed by AWS	Fully managed by AWS	You manage (kubeadm, kops)
Control Plane Cost	$0.10/hr per cluster	Free	EC2 cost for master nodes
Learning Curve	High (K8s knowledge needed)	Low	Very High
Portability	High (standard K8s)	Low (AWS-only)	High (standard K8s)
Ecosystem	Vast (Helm, Istio, Argo CD)	AWS services only	Vast
Serverless Option	Fargate Profiles	Fargate (better support)	Not available
Upgrade Complexity	Low (managed)	Lowest	Very High
Windows Containers	Yes	Yes	Yes
GPU Support	Yes (managed node group)	Yes (EC2 launch type)	Yes
Best For	Complex multi-team, K8s expertise, portable workloads	AWS-native, simple, Fargate-first	Absolute control, unusual requirements

8. Master Comparison — ECR, ECS, EKS Together

Service	Layer	What It Does	Without It You Would...
Amazon ECR	Image Storage	Store, version, secure, and replicate container images	Manage your own Docker registry server
Amazon ECS	Orchestration (AWS-native)	Run, scale, and update containers; manage tasks and services	Manually SSH into servers and run docker commands
Amazon EKS	Orchestration (Kubernetes)	Managed K8s control plane; run pods on nodes or Fargate	Run and patch your own K8s master nodes on EC2

How They Work Together (Typical Architecture)

1. Developer writes code and creates a Dockerfile
2. CI/CD (CodePipeline / GitHub Actions) builds the Docker image
3. Image pushed to Amazon ECR (private registry in your account)
4. ECR scans the image for vulnerabilities automatically
5. ECS or EKS deployment triggered — pulls image from ECR

ECS: update Task Definition, update Service -> rolling deploy

EKS: update Deployment image tag -> kubectl apply -> rolling update

6. ALB distributes traffic to healthy containers
7. CloudWatch collects logs and metrics; alarms trigger auto-scaling

ECR = WHERE images live | ECS/EKS = HOW images run at scale

9. Decision Guide — Which Container Service?

Your Situation	Recommended Service
Store and manage Docker images privately on AWS	Amazon ECR
Run containers on AWS, small team, no K8s experience	Amazon ECS + Fargate
AWS-native microservices, already using ALB and IAM	Amazon ECS
Need Kubernetes APIs, Helm charts, Istio, or K8s RBAC	Amazon EKS
Multi-cloud portability or on-premises K8s workloads	Amazon EKS
Complex multi-team with hundreds of microservices	Amazon EKS
Serverless containers — no infrastructure to manage at all	ECS Fargate
GPU / ML training workloads on Kubernetes	Amazon EKS + GPU Node Group
Existing K8s workload migration from on-prem	Amazon EKS
Simple batch jobs / event-driven containers (triggered, run, terminate)	Amazon ECS Fargate Spot